

Python Interview Questions Part 1 - Complete Answer Handbook

Simple, interview-focused explanations for all questions from the poster.

Python Basics

Q: What is Python?

Python is a high-level, interpreted programming language known for its simple syntax and readability. It allows developers to build applications quickly and is widely used in web development, automation, data science, and AI.

Q: Why is Python popular?

Python is easy to learn and has a large ecosystem of libraries and frameworks. Its clean syntax helps developers write and maintain code efficiently.

Q: Features of Python?

Python supports object-oriented programming, dynamic typing, portability, and a huge standard library. It is also open source and platform independent.

Q: Python 2 vs Python 3?

Python 3 is the latest version and includes better Unicode support, improved syntax, and security updates. Python 2 is no longer officially supported.

Q: How is Python different?

Python emphasizes code readability and rapid development. Compared to many languages, it requires less code to perform the same task.

Data Types & Variables

Q: Built-in data types?

Common data types include int, float, str, bool, list, tuple, set, and dictionary. Each type is used for storing different forms of data.

Q: Mutable vs immutable?

Mutable objects can be modified after creation, such as lists and dictionaries. Immutable objects like strings and tuples cannot be changed.

Q: Type casting?

Type casting converts data from one type to another using functions such as int(), float(), and str(). It helps ensure compatibility during operations.

Q: Check variable type?

Use the type() function to determine the data type of a variable. It returns the class of the object.

Q: == vs is?

`==` compares values, whereas `is` compares object identities in memory. Two objects can be equal but not identical.

Operators

Q: Types of operators?

Python provides arithmetic, comparison, logical, assignment, membership, identity, and bitwise operators. They are used to perform various operations on data.

Q: / vs //?

The `/` operator returns normal division with decimals. The `//` operator performs floor division and removes the fractional part.

Q: Modulo operator?

The modulus operator (`%`) returns the remainder after division. It is often used for checking divisibility.

Q: Operator precedence?

Operator precedence determines the order in which operators are evaluated. Parentheses can be used to control evaluation order.

Q: Identity operators?

The identity operators are `is` and `is not`. They check whether two references point to the same object.

Control Flow

Q: Use of if, elif, else?

These statements are used for decision-making in Python. They allow different blocks of code to execute based on conditions.

Q: How does a for loop work?

A `for` loop iterates through sequences such as lists, tuples, strings, or ranges. It executes a block of code for each item.

Q: How does a while loop work?

A `while` loop continues executing as long as its condition remains true. It is useful when the number of iterations is unknown.

Q: break, continue, pass?

`break` exits a loop immediately, `continue` skips the current iteration, and `pass` acts as a placeholder statement.

Q: range() function?

`range()` generates a sequence of numbers. It is commonly used with loops.

Strings

Q: What is a string?

A string is a sequence of characters enclosed in quotes. Strings are immutable in Python.

Q: How to declare a string?

Strings can be declared using single quotes, double quotes, or triple quotes. Triple quotes support multi-line strings.

Q: String slicing and indexing?

Indexing accesses a single character, while slicing extracts a portion of the string using start and end positions.

Q: Raw strings?

Raw strings treat backslashes as literal characters. They are commonly used for file paths and regular expressions.

Q: f-strings?

f-strings allow variables and expressions to be embedded directly inside strings. They provide readable formatting.

Q: Concatenate strings?

Strings can be joined using the + operator or the join() method. This combines multiple strings into one.

Lists

Q: What is a list?

A list is an ordered and mutable collection of elements. It can store multiple data types.

Q: Create a list?

Lists are created using square brackets. Example: [1, 2, 3].

Q: List slicing and indexing?

Indexing retrieves specific elements, while slicing extracts a subset of the list.

Q: Add, remove, update elements?

Use append(), insert(), remove(), pop(), and indexing operations to modify lists.

Q: List comprehensions?

List comprehensions provide a concise way to create lists using a single line of code.

Tuples

Q: What is a tuple?

A tuple is an ordered and immutable collection of items. Once created, it cannot be modified.

Q: Tuple vs list?

Lists are mutable, whereas tuples are immutable. Tuples are generally faster and safer for fixed data.

Q: Can we modify a tuple?

No, tuples cannot be directly modified after creation. A new tuple must be created instead.

Q: Tuple unpacking?

Tuple unpacking assigns tuple values to multiple variables in a single statement.

Q: When should we use tuples?

Use tuples when data should remain unchanged and integrity is important.

Sets

Q: What is a set?

A set is an unordered collection of unique elements. Duplicate values are automatically removed.

Q: Set vs list?

Lists allow duplicates and maintain order, whereas sets store only unique values.

Q: Set operations?

Common operations include union, intersection, difference, and symmetric difference.

Q: union(), intersection(), difference()?

These methods combine sets, find common elements, and identify unique elements respectively.

Q: Can sets have duplicates?

No. Sets automatically eliminate duplicate values.

Dictionaries

Q: What is a dictionary?

A dictionary stores data as key-value pairs. Keys must be unique.

Q: Create a dictionary?

Dictionaries are created using curly braces or the dict() function.

Q: Add, update, delete items?

Items can be added or updated using keys and removed using pop() or del.

Q: Dictionary methods?

Popular methods include keys(), values(), items(), get(), update(), and pop().

Q: get() vs []?

get() safely returns a value or default value, while [] raises a KeyError if the key is missing.

Functions

Q: What is a function?

A function is a reusable block of code designed to perform a specific task.

Q: How do you define a function?

Functions are defined using the `def` keyword followed by a function name.

Q: Parameters and arguments?

Parameters are variables in the function definition, while arguments are actual values passed during function calls.

Q: *args vs **kwargs?

`*args` collects positional arguments and `**kwargs` collects keyword arguments.

Q: Lambda function?

A lambda function is a small anonymous function defined in a single expression.

OOP Concepts

Q: What is OOP?

Object-Oriented Programming organizes code using classes and objects. It improves reusability and maintainability.

Q: Four pillars of OOP?

Encapsulation, Inheritance, Polymorphism, and Abstraction are the four pillars.

Q: Class and object?

A class is a blueprint, while an object is an instance of that class.

Q: __init__() in Python?

It is a constructor method that initializes object attributes when an object is created.

Q: Inheritance?

Inheritance allows a class to acquire properties and methods from another class.

Q: Polymorphism?

Polymorphism allows the same interface to behave differently for different objects.

Q: Encapsulation?

Encapsulation restricts direct access to data and protects object integrity.

Q: Abstraction?

Abstraction hides implementation details and shows only essential functionality.

Exception Handling

Q: What is an exception?

An exception is a runtime error that interrupts normal program execution.

Q: Why use exception handling?

It prevents program crashes and allows graceful error recovery.

Q: try, except, else, finally?

These blocks are used to catch, handle, and clean up after exceptions.

Q: raise vs assert?

raise explicitly triggers an exception, while assert validates a condition during debugging.

Q: Custom exceptions?

Custom exceptions are user-defined exception classes created by inheriting Exception.

Modules & Packages

Q: What is a module?

A module is a Python file containing functions, variables, and classes.

Q: Import a module?

Modules are imported using the import statement.

Q: Module vs package?

A module is a single file, whereas a package is a collection of related modules.

Q: __name__ == '__main__'?

This condition checks whether a file is being run directly or imported.

Q: Create your own module?

Write Python code in a .py file and import it where needed.

File Handling

Q: How do you open a file?

Files are opened using the open() function with an appropriate mode.

Q: Different file modes?

Common modes include r, w, a, x, rb, and wb.

Q: Read and write to a file?

Use read() to read data and write() to write data into files.

Q: Use of with open()?

It automatically closes the file after use and prevents resource leaks.

Q: Handle file-not-found exceptions?

Use try-except blocks to catch FileNotFoundError.

Advanced Concepts

Q: What is a decorator?

A decorator modifies or extends the behavior of a function without changing its source code.

Q: What is a generator?

A generator produces values one at a time using yield, saving memory.

Q: What is yield?

yield returns a value and pauses function execution until the next call.

Q: What is a context manager?

A context manager handles resource setup and cleanup automatically, usually with the with statement.

Q: Iterators and iterables?

Iterables are objects that can be traversed, while iterators provide one element at a time.

Q: What is GIL?

The Global Interpreter Lock allows only one thread to execute Python bytecode at a time in CPython.